# 3D WEBSOFT INC. FOUNDATION

## *FUTURE TECH SCHOOL PROGRAM*

***Faculty Name :***

***Course Duration : 5 Months***

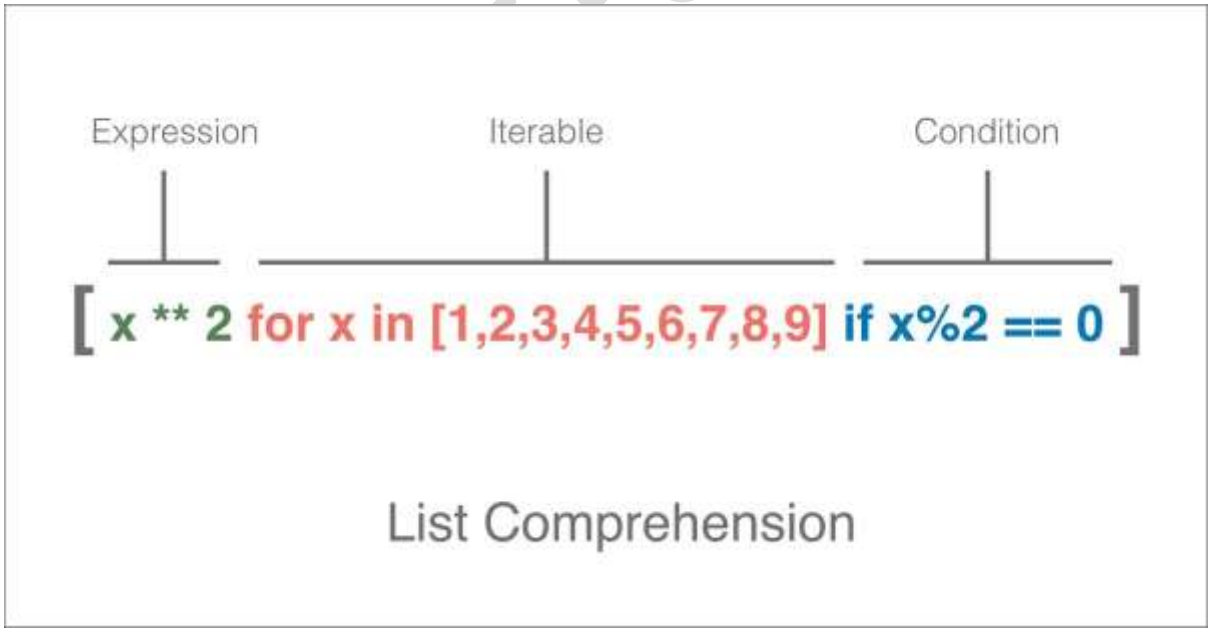
***Classes : Offline & Online***

***Daily Teaching : 2 Hours***

***Total Classes : 100***

## *Level 4 – AI Professional Foundation(11th–12th Standard)*

## *Class 8 – List, Set & Dictionary Comprehension*




## *Learning Objectives*

By the end of this lesson, students will be able to:

- Understand what **comprehension** means in Python.
- Create lists using **list comprehension**.
- Create sets using **set comprehension**.
- Create dictionaries using **dictionary comprehension**.
- Use conditions inside comprehensions.
- Transform existing data.
- Filter data based on conditions.
- Process student marks, numbers, names, and prices.
- Understand when comprehensions should and should not be used.
- Build small real-world data-processing programs.

**1. What is Comprehension?**

A **comprehension** is a short and powerful way to create a new collection from existing data.

Normally, we might use a for loop:

```
numbers = [1, 2, 3, 4, 5]


squares = []


for number in numbers:
    squares.append(number ** 2)


print(squares)
```

Output:

```
[1, 4, 9, 16, 25]
```

3DWEBSOFT FOUNDATION

Using list comprehension:

numbers = [1, 2, 3, 4, 5]

squares = [number ** 2 for number in numbers]

print(squares)

Same result:

[1, 4, 9, 16, 25]

**Basic idea**

Existing Data

↓

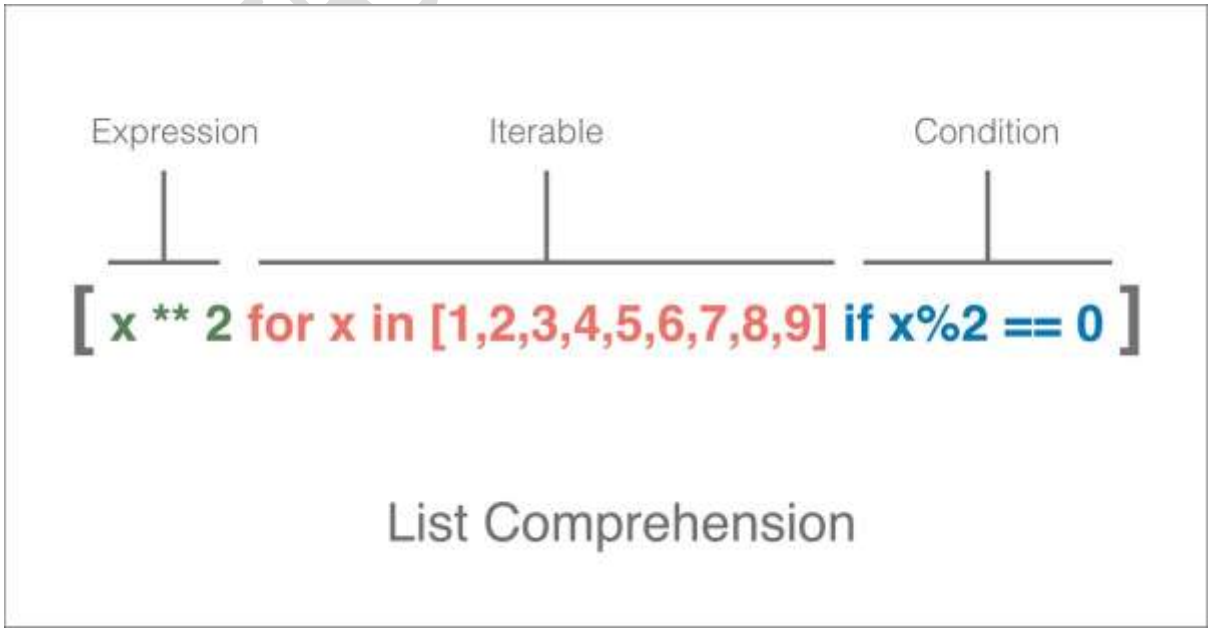
Process / Transform

↓

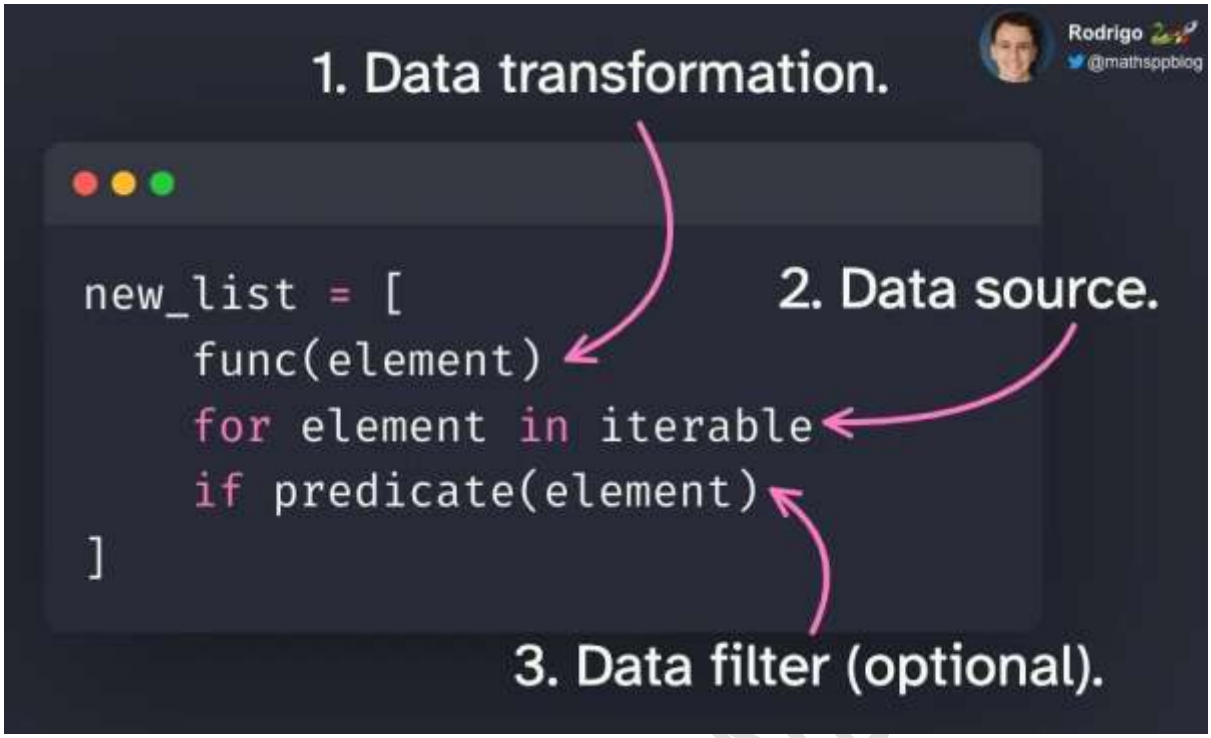
Condition

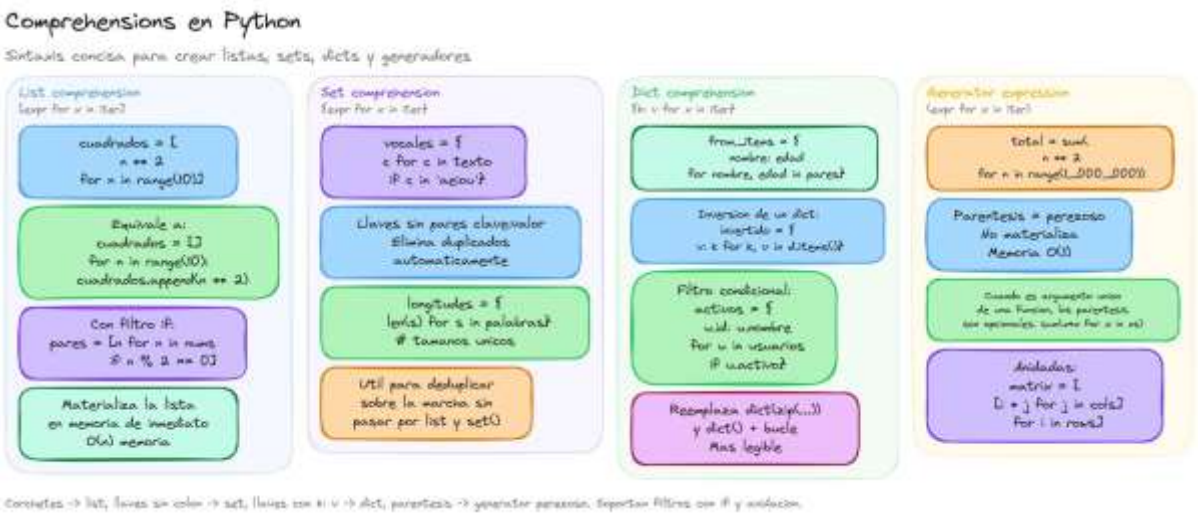
↓

New Collection




RSOFT FOUNDATION




## 2. Why Use Comprehensions?

Comprehensions can make data-processing code:

- Shorter
- Easier to read when simple
- Faster to write
- Useful for transforming data

- Useful for filtering data

For example:

**Traditional approach**

```
even_numbers = []


for number in range(1, 11):
    if number % 2 == 0:
        even_numbers.append(number)
```

**Comprehension**

```
even_numbers = [number for number in range(1, 11) if number % 2 == 0]
```

## *PART A – LIST COMPREHENSION*

**3. What is List Comprehension?**

**List comprehension** creates a new list from an iterable such as a list, tuple, string, or range().

**Basic Syntax**

```
[expression for item in iterable]
```

Example:

```
numbers = [1, 2, 3, 4, 5]


squares = [x ** 2 for x in numbers]


print(squares)
```

Output:

```
[1, 4, 9, 16, 25]
```

3DWEBSOFT FOUNDATION

**4. Understanding the Syntax**

Consider:

squares = [x ** 2 for x in numbers]

Break it down:

[x ** 2 for x in numbers]

↓ ↓ ↓

expression item collection

Meaning:

Take every x from numbers, calculate $x^2$, and put the result into a new list.

**5. List Comprehension with range()**

numbers = [x for x in range(1, 11)]

print(numbers)

Output:

[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

**6. Generate Squares**

squares = [x ** 2 for x in range(1, 11)]

print(squares)

Output:

[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]

**7. Generate Cubes**

3DWEBSOFT FOUNDATION

```
cubes = [x ** 3 for x in range(1, 6)]


print(cubes)
```

Output:

```
[1, 8, 27, 64, 125]
```

**8. List Comprehension with Strings**

```
names = ["aisha", "rahul", "priya", "arjun"]


capital_names = [name.upper() for name in names]


print(capital_names)
```

Output:

```
['AISHA', 'RAHUL', 'PRIYA', 'ARJUN']
```

This is an example of **data transformation**.

**9. List Comprehension with a Condition**

We can filter data using if.

**Syntax**

```
[expression for item in iterable if condition]
```

Example:

```
numbers = [1, 2, 3, 4, 5, 6]


even_numbers = [x for x in numbers if x % 2 == 0]
```

3DWEBSOFT FOUNDATION

print(even_numbers)

Output:

[2, 4, 6]

**Processing flow**

1 → odd → ignore

2 → even → keep

3 → odd → ignore

4 → even → keep

5 → odd → ignore

6 → even → keep

**10. Filter Student Marks**

Suppose:

marks = [35, 67, 89, 42, 28, 95, 76]

Find marks greater than or equal to 40:

passed = [mark for mark in marks if mark >= 40]

print(passed)

Output:

[67, 89, 42, 95, 76]

**11. Find Failed Marks**

failed = [mark for mark in marks if mark < 40]

print(failed)

3DWEBSOFT FOUNDATION

Output:

[35, 28]

**12. Add Extra Marks**

Suppose every student receives 5 bonus marks.

marks = [70, 80, 65, 90]

updated_marks = [mark + 5 for mark in marks]

print(updated_marks)

Output:

[75, 85, 70, 95]

This is another example of transformation.

**13. Conditional Expression in List Comprehension**

We can use if...else inside a comprehension.

**Syntax**

[expression_if_true if condition else expression_if_false for item in iterable]

Example:

marks = [85, 35, 72, 28, 90]

result = ["Pass" if mark >= 40 else "Fail" for mark in marks]

print(result)

Output:

3DWEBSOFT FOUNDATION

['Pass', 'Fail', 'Pass', 'Fail', 'Pass']

Notice the difference:

**Filtering**

[mark for mark in marks if mark >= 40]

Only selected values are returned.

**Conditional transformation**

["Pass" if mark >= 40 else "Fail" for mark in marks]

Every value produces a result.

**14. Convert Celsius to Fahrenheit**

Formula:

F = (C × 9/5) + 32

Python:

```
celsius = [0, 10, 20, 30, 40]

fahrenheit = [(c * 9/5) + 32 for c in celsius]

print(fahrenheit)
```

Output:

[32.0, 50.0, 68.0, 86.0, 104.0]

**15. String Processing with List Comprehension**

```
words = ["python", "machine", "learning", "ai"]

long_words = [word for word in words if len(word) > 4]
```

3DWEBSOFT FOUNDATION

print(long_words)

Output:

['python', 'machine', 'learning']

## *PART B – SET COMPREHENSION*

**16. What is Set Comprehension?**

Set comprehension creates a **set** using a compact syntax.

**Syntax**

{expression for item in iterable}

Example:

numbers = [1, 2, 2, 3, 4, 4, 5]

unique_squares = {x ** 2 for x in numbers}

print(unique_squares)

The result contains unique values.

**17. Why Set Comprehension?**

Sets automatically remove duplicates.

Example:

numbers = [1, 2, 2, 3, 3, 4]

unique_numbers = {x for x in numbers}

3DWEBSOFT FOUNDATION

print(unique_numbers)

Conceptually:

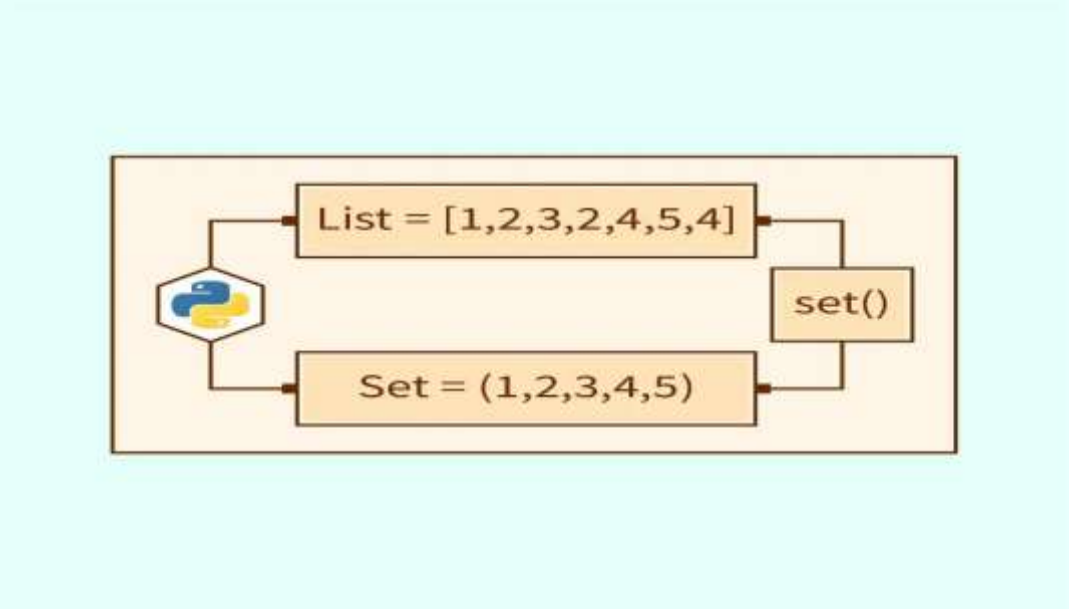
Input:

1 2 2 3 3 4

↓

Set:

1 2 3 4




## 3.4 What is Set Comprehension?

- Set comprehensions are pretty similar to list comprehensions.
- The only difference between them is that set comprehensions use curly brackets { }.
- No Duplicate Values Allowed

**Syntax:** {expression for item in iterable if conditional}

**Example 1: Create A List Of Odd Numbers From 0 to 20.**

**Common Way:**

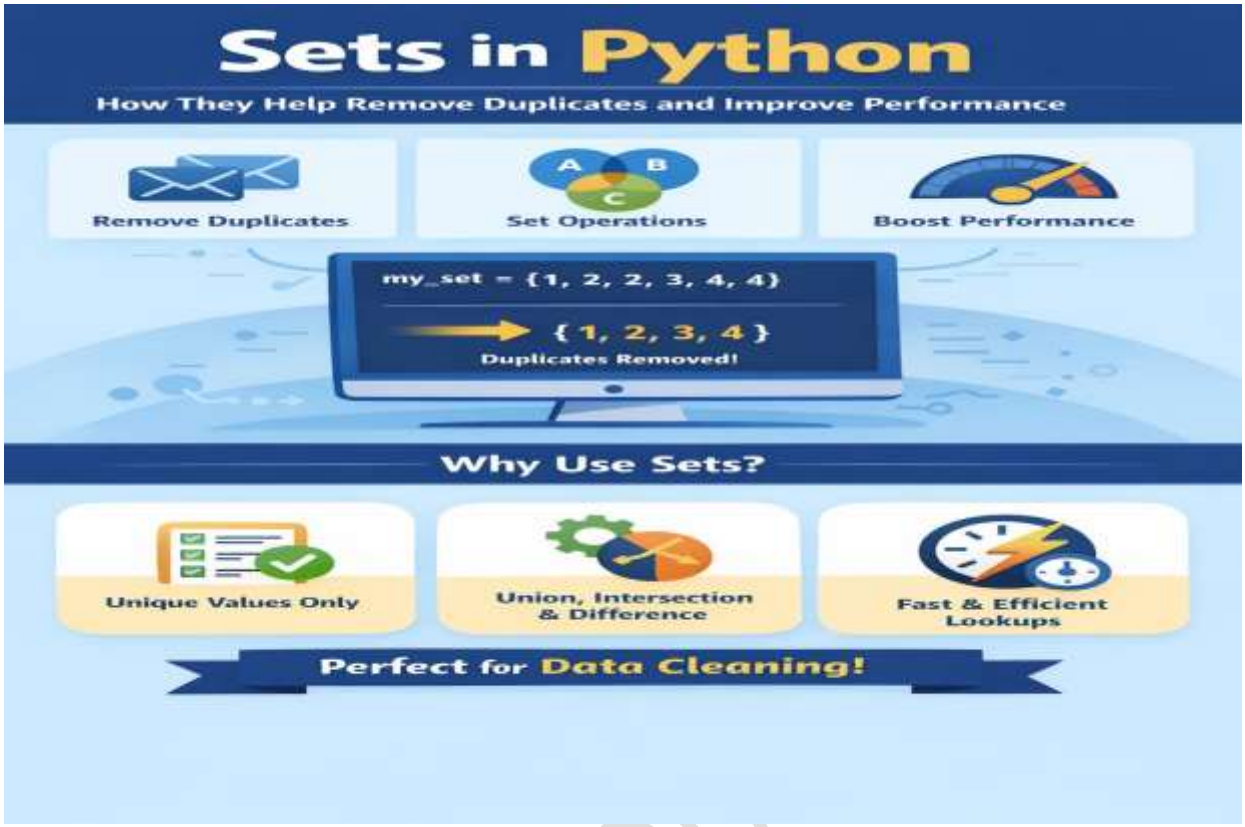
```
s1 = set()
for i in range(20):
    if i%2 != 0:
        s1.add(i)
print(s1)
```

**Output:**
{1, 3, 5, 7, 9, 11, 13, 15, 17, 19}

**Using Set Comprehension:**

```
s1 = {i for i in range(20) if i%2 != 0}
print(s1)
```

**Output:**
{1, 3, 5, 7, 9, 11, 13, 15, 17, 19}




**18. Find Unique First Letters**

names = ["Aisha", "Arjun", "Rahul", "Ravi", "Priya"]

initials = {name[0] for name in names}

print(initials)

Possible output:

{'A', 'R', 'P'}

Notice that "Aisha" and "Arjun" both produce A, but the set keeps it only once.

**19. Set Comprehension with Conditions**

numbers = range(1, 11)

3DWEBSOFT FOU

```
even_numbers = {x for x in numbers if x % 2 == 0}


print(even_numbers)
```

Result:

{2, 4, 6, 8, 10}

# *PART C – DICTIONARY COMPREHENSION*

**20. What is Dictionary Comprehension?**

Dictionary comprehension creates a new dictionary using a compact syntax.

**Syntax**

{key: value for item in iterable}

Example:

```
numbers = [1, 2, 3, 4, 5]


squares = {x: x ** 2 for x in numbers}


print(squares)
```

Output:

{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

**21. Understanding Dictionary Comprehension**

```
squares = {x: x ** 2 for x in numbers}
```

Means:

x → key

$x^2$ → value

3DWEBSOFT FOUNDATION

So:

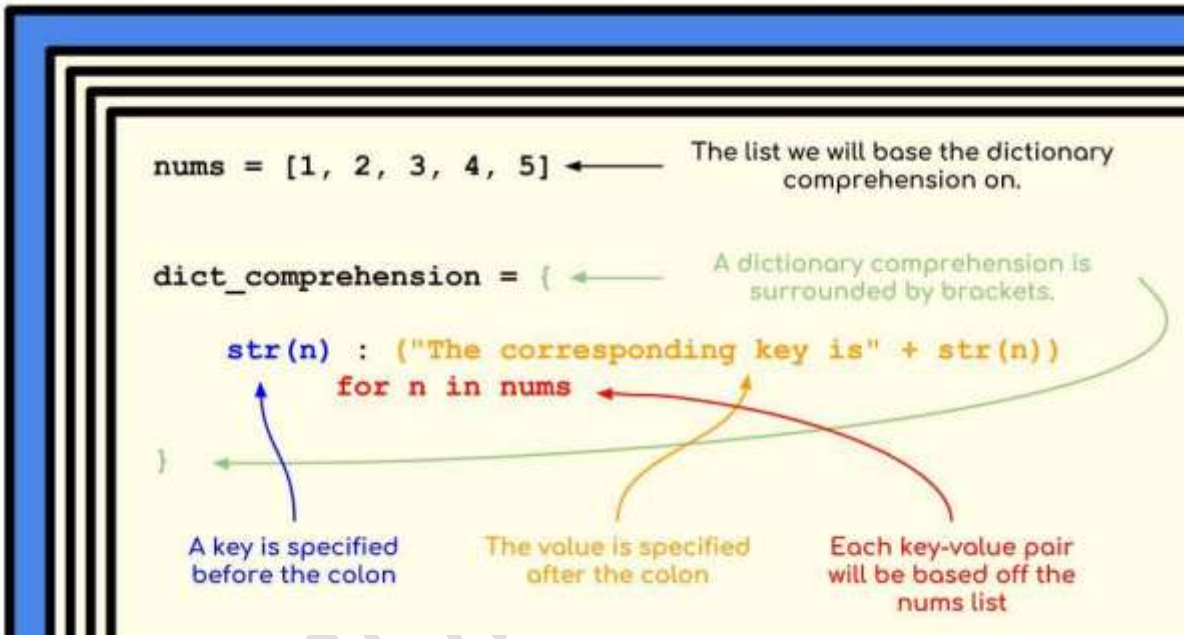
1 → 1

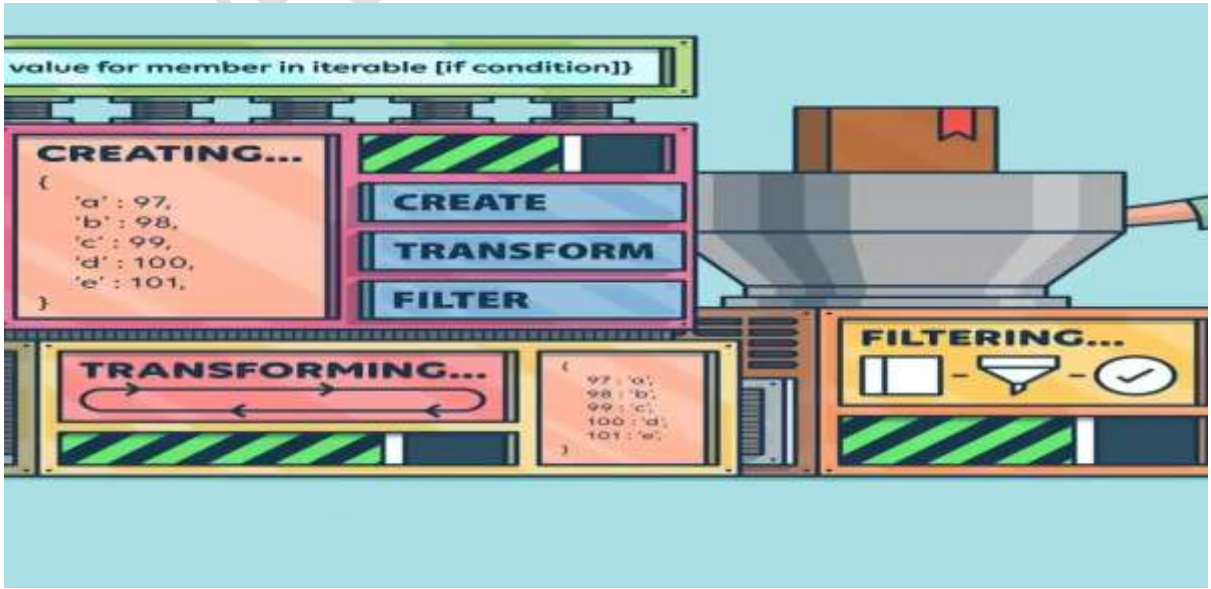
2 → 4

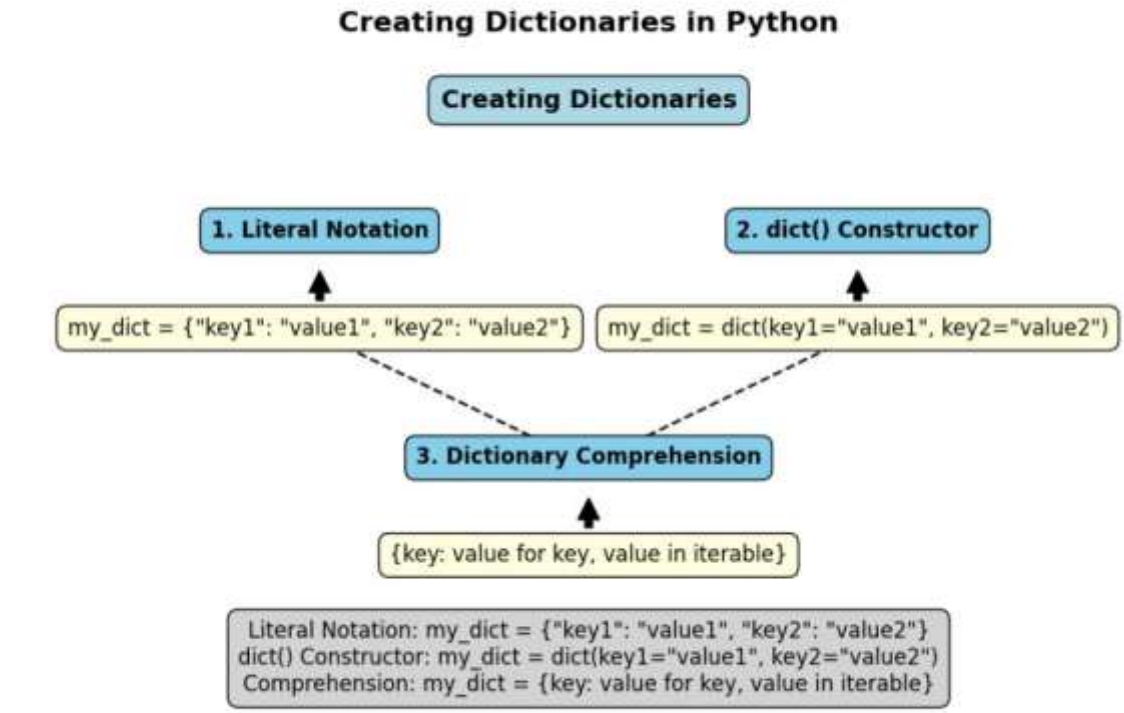
3 → 9

4 → 16

5 → 25






## *22. Student Marks Dictionary*

Suppose:

```
students = {
    "Aisha": 85,
    "Rahul": 35,
    "Priya": 92,
    "Arjun": 78
}
```

Find students who passed:

```
passed = {
    name: mark
    for name, mark in students.items()
```

3DWEBSOFT

```
    if mark >= 40
}

print(passed)
```

Output:

```
{'Aisha': 85, 'Priya': 92, 'Arjun': 78}
```

**23. Add 5 Bonus Marks**

```
marks = {
    "Aisha": 85,
    "Rahul": 70,
    "Priya": 90
}

updated_marks = {
    name: mark + 5
    for name, mark in marks.items()
}

print(updated_marks)
```

Output:

```
{'Aisha': 90, 'Rahul': 75, 'Priya': 95}
```

**24. Create a Grade Dictionary**

```
marks = {
```

3DWEBSOFT FOUNDATION

```
    "Aisha": 95,
    "Rahul": 72,
    "Priya": 88,
    "Arjun": 45
}

grades = {
    name: (
        "A+" if mark >= 90 else
        "A" if mark >= 80 else
        "B" if mark >= 70 else
        "C"
    )
    for name, mark in marks.items()
}

print(grades)
```

Output:

```
{
    'Aisha': 'A+',
    'Rahul': 'B',
    'Priya': 'A',
    'Arjun': 'C'
}
```

3DWEBSOFT FOUNDATION

For beginners, it is often clearer to calculate complex grades using a normal function and then use that function in the comprehension.

# *Practical Session*

## *PRACTICAL 1 – Student Marks Processing*

**Problem**

Given:

marks = [45, 67, 89, 32, 76, 91, 28]

Create:

1. List of passed marks
2. List of failed marks
3. List of marks after adding 5 bonus marks

**Solution**

```
marks = [45, 67, 89, 32, 76, 91, 28]


passed = [mark for mark in marks if mark >= 40]


failed = [mark for mark in marks if mark < 40]


bonus_marks = [mark + 5 for mark in marks]


print("Passed:", passed)

print("Failed:", failed)

print("Bonus Marks:", bonus_marks)
```

3DWEBSOFT FOUNDATION

## *PRACTICAL 2 – Student Names Processing*

```
students = ["aisha", "rahul", "priya", "arjun", "sara"]


upper_names = [name.upper() for name in students]


name_lengths = [len(name) for name in students]


print("Uppercase Names:", upper_names)
print("Name Lengths:", name_lengths)
```

## *PRACTICAL 3 – Filter High Scorers*

```
marks = [65, 82, 91, 74, 95, 58, 88]


high_scorers = [mark for mark in marks if mark >= 80]


print("High Scorers:", high_scorers)
```

Output:

High Scorers: [82, 91, 95, 88]

## *PRACTICAL 4 – Data Cleaning Using Set Comprehension*

Suppose student IDs were entered multiple times:

```
student_ids = [101, 102, 101, 103, 104, 102, 105]
```

Create unique IDs:

```
unique_ids = {student_id for student_id in student_ids}
```

3DWEBSOFT FOUNDATION

```
print(unique_ids)
```

## *PRACTICAL 5 – Dictionary Comprehension for Student Database*

```
students = {
   "Aisha": 85,
   "Rahul": 72,
   "Priya": 94,
   "Arjun": 38
}

passed_students = {
   name: mark
   for name, mark in students.items()
   if mark >= 40
}

print("Passed Students:")
print(passed_students)
```

## *PRACTICAL 6 – Create a Square Dictionary*

```
numbers = range(1, 11)

square_dictionary = {
   number: number ** 2
```

3DWEBSOFT FOUNDATION

```
    for number in numbers
}


print(square_dictionary)
```

Output:

```
{
 1: 1,
 2: 4,
 3: 9,
 4: 16,
 5: 25,
 6: 36,
 7: 49,
 8: 64,
 9: 81,
 10: 100
}
```

## *PRACTICAL 7 – Product Price Processing*

Suppose a shop has:

```
prices = {
    "Laptop": 50000,
    "Mouse": 800,
    "Keyboard": 1500,
    "Monitor": 12000
```

3DWEBSOFT FOUNDATION

```
}
```

Apply a 10% discount:

```
discounted_prices = {
    product: price * 0.90
    for product, price in prices.items()
}


print(discounted_prices)
```

This demonstrates how dictionary comprehension can process structured data.

## *PRACTICAL 8 – Student Performance Analyzer*

Now combine lists, sets, and dictionaries.

```
students = {
    "Aisha": 92,
    "Rahul": 35,
    "Priya": 88,
    "Arjun": 76,
    "Sara": 35
}
```

**Step 1 – Passed Students**

```
passed = {
    name: mark
    for name, mark in students.items()
    if mark >= 40
}
```

3DWEBSOFT FOUNDATION

**Step 2 – Failed Students**

```
failed = {
    name: mark
    for name, mark in students.items()
    if mark < 40
}
```

**Step 3 – High Scorers**

```
top_students = {
    name: mark
    for name, mark in students.items()
    if mark >= 80
}
```

**Step 4 – Display**

```
print("Passed:", passed)
print("Failed:", failed)
print("High Scorers:", top_students)
```

## *Mini Project – Data Processing Dashboard*

Create a small Python program that processes student data.

**Input**

```
students = {
    "Aisha": 92,
    "Rahul": 35,
    "Priya": 88,
    "Arjun": 76,
```

3DWEBSOFT FOUNDATION

```
    "Sara": 95,
    "John": 42
}
```

**Requirements**

Your program should create:

**1. Passed students**

```
passed = {
    name: mark
    for name, mark in students.items()
    if mark >= 40
}
```

**2. Failed students**

```
failed = {
    name: mark
    for name, mark in students.items()
    if mark < 40
}
```

**3. Top performers**

```
top = {
    name: mark
    for name, mark in students.items()
    if mark >= 80
}
```

**4. Student names**

```
names = [name for name in students]
```

3DWEBSOFT FOUNDATION

**5. Unique mark values**

unique_marks = {mark for mark in students.values()}

**Final Display**

===== STUDENT DATA DASHBOARD =====

All Students:

Aisha → 92

Rahul → 35

Priya → 88

Arjun → 76

Sara → 95

John → 42

Passed Students:

Aisha → 92

Priya → 88

Arjun → 76

Sara → 95

John → 42

Failed Students:

Rahul → 35

Top Performers:

Aisha → 92

3DWEBSOFT FOUNDATION

Priya → 88

Sara → 95

## *Comprehension vs Normal Loop*

**Normal Loop**

numbers = [1, 2, 3, 4, 5]

squares = []

for number in numbers:

    squares.append(number ** 2)

**List Comprehension**

numbers = [1, 2, 3, 4, 5]

squares = [number ** 2 for number in numbers]

Both are valid.

**Important Rule**

**Do not use comprehensions just to make code shorter.**

If a comprehension becomes difficult to understand, use a normal loop or function.

## *Common Mistakes*

**Mistake 1 – Forgetting the expression**

Incorrect:

[x for in numbers]

3DWEBSOFT FOUNDATION

Correct:

[x for x in numbers]

**Mistake 2 – Incorrect condition**

Incorrect:

[x if x > 50 for x in marks]

For filtering, write:

[x for x in marks if x > 50]

**Mistake 3 – Confusing List and Set Comprehension**

List:

[x for x in numbers]

Set:

{x for x in numbers}

Dictionary:

{x: x**2 for x in numbers}

## *Comparison Table*

| Type | Syntax | Result |
|---|---|---|
| List comprehension | [expression for item in iterable] | List |
| Set comprehension | {expression for item in iterable} | Set |
| Dictionary comprehension | {key: value for item in iterable} | Dictionary |

**Real-World Applications**

3DWEBSOFT FOUNDATION

Comprehensions are useful in:

**Data Science**

Filtering and transforming datasets.

**Machine Learning**

Preparing training data.

**Data Cleaning**

Removing unwanted or duplicate information.

**E-Commerce**

Applying discounts to product prices.

**Education**

Filtering student marks and generating reports.

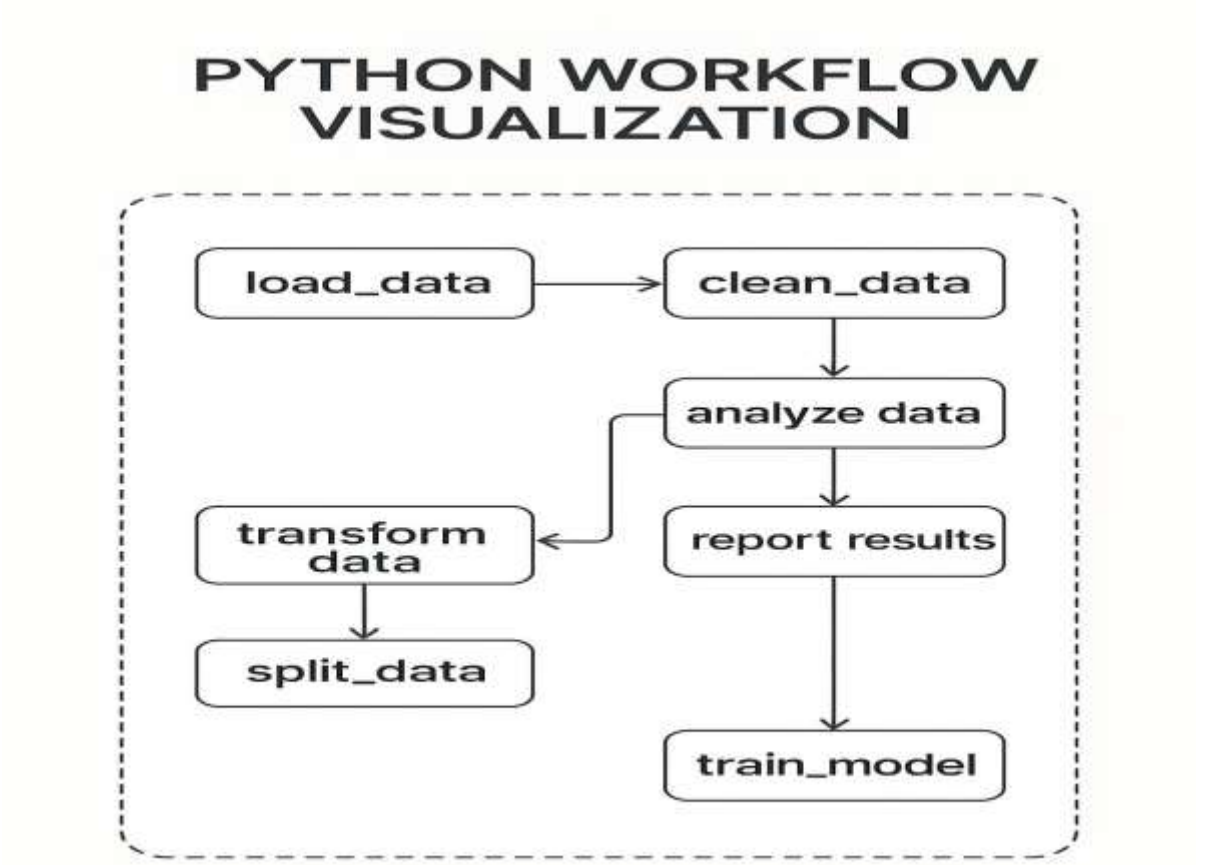
**Search Systems**

Filtering matching records.

**Business Analytics**

Transforming and organizing data.

3DWEBSOFT FOUNDATION



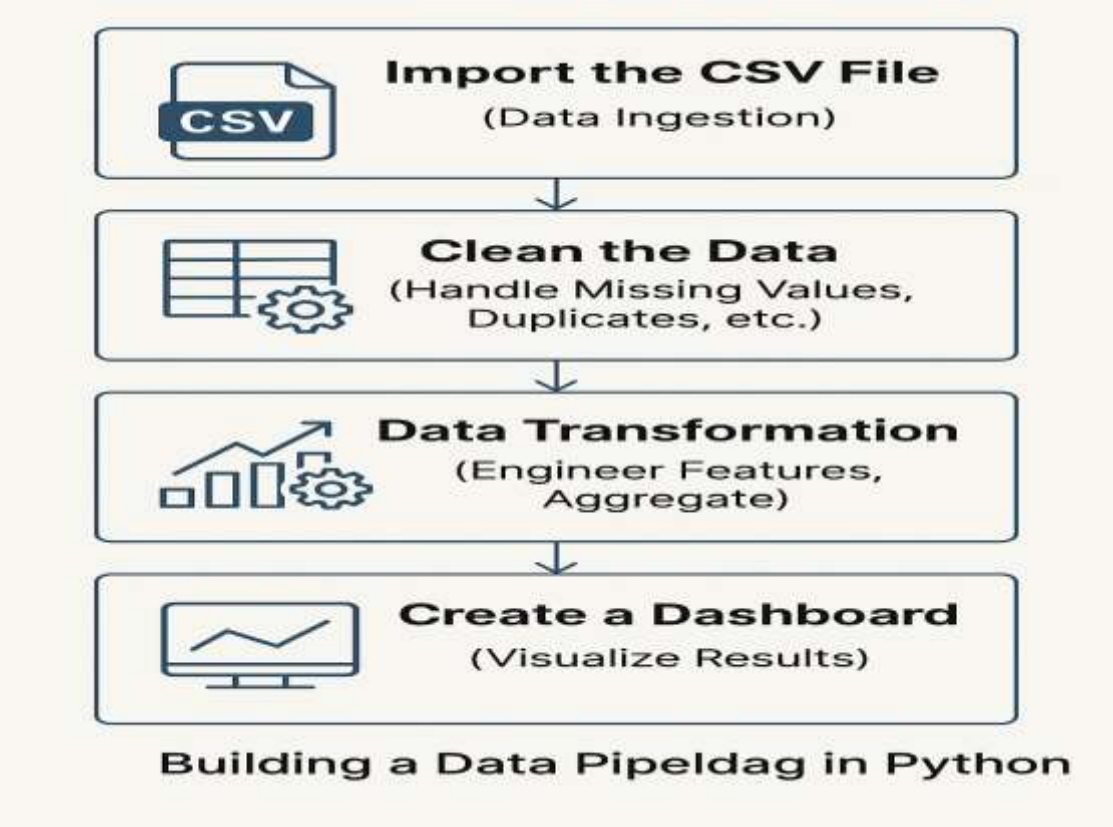

```python
#3)Data Cleaning:
import pandas as pd

# Sample data
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eva'],
    'Age': [20, 22, None, 21, 23],
    'Grade': ['A', 'B', 'C', 'A', 'B']
}

students = pd.DataFrame(data)

# Remove rows with missing values and reset index
students_cleaned = students.dropna().reset_index(drop=True)
print(students_cleaned)
```

```
    Name   Age Grade
0  Alice  20.0     A
1    Bob  22.0     B
2  David  21.0     A
3    Eva  23.0     B
```




## *Interactive Classroom Activity – "Comprehension Challenge"*

Give students:

numbers = [5, 10, 15, 20, 25, 30]

Ask them to create:

**Challenge 1**

Squares of all numbers.

**Challenge 2**

Only numbers greater than 15.

**Challenge 3**

Numbers divided by 5.

**Challenge 4**

A set containing unique remainders when numbers are divided by 4.

3DWEBSOFT FOU

**Challenge 5**

A dictionary where each number maps to its square.

Expected concepts:

[x ** 2 for x in numbers]

[x for x in numbers if x > 15]

[x / 5 for x in numbers]

{x % 4 for x in numbers}

{x: x ** 2 for x in numbers}

## *Debugging Activity*

Ask students to identify the error:

marks = [40, 55, 70, 25, 90]

passed = [mark for marks if mark >= 40]

print(passed)

**Error**

The variable mark was not introduced in the comprehension.

**Correct code**

passed = [mark for mark in marks if mark >= 40]

3DWEBSOFT FOUNDATION

# *Quiz*

**1. What is comprehension?**

**Answer:** A concise way to create a new collection from existing data.

**2. Which brackets are used for list comprehension?**

**Answer:** []

**3. Which brackets are used for set comprehension?**

**Answer:** {}

**4. What is the structure of dictionary comprehension?**

**Answer:**

{key: value for item in iterable}

**5. How can we filter values in a comprehension?**

**Answer:** By using an if condition.

**6. Write a comprehension to create squares from 1 to 5.**

[x ** 2 for x in range(1, 6)]

**7. Write a comprehension to find even numbers.**

[x for x in numbers if x % 2 == 0]

**8. Why is set comprehension useful for duplicate data?**

**Answer:** Sets store only unique values.

**9. What does .items() provide for a dictionary?**

**Answer:** Key-value pairs.

**10. Should every loop be converted into a comprehension?**

**Answer:** No. Comprehensions should be used when they improve readability and clarity.

3DWEBSOFT FOUNDATION

# *Homework*

**Task 1 – Number Processing**

Given:

numbers = [10, 15, 20, 25, 30, 35, 40]

Create:

1. Squares
2. Cubes
3. Even numbers
4. Numbers greater than 25

**Task 2 – Student Marks**

marks = [35, 55, 72, 89, 28, 95, 67]

Create:

- Passed marks
- Failed marks
- Marks above 80
- Marks after adding 5 bonus marks

**Task 3 – Dictionary**

Create a dictionary:

```
students = {
   "Aisha": 85,
   "Rahul": 38,
   "Priya": 92,
   "Arjun": 76
}
```

3DWEBSOFT FOUNDATION

Using dictionary comprehension, create:

- Passed students
- Failed students
- Students scoring 80 or above

## *2-Hour Class Plan*

| Time | Activity |
|---|---|
| 0–10 min | Revision of Lists, Sets & Dictionaries |
| 10–25 min | Introduction to Comprehensions |
| 25–45 min | List Comprehension |
| 45–60 min | Conditions & Data Filtering |
| 60–75 min | Set Comprehension |
| 75–90 min | Dictionary Comprehension |
| 90–105 min | Student Data Processing |
| 105–112 min | Debugging Challenge |
| 112–118 min | Quiz |
| 118–120 min | Homework & Recap |

3DWEBSOFT FOUNDATION

## *Quick Reference*

**List**

[x for x in data]

**List with condition**

[x for x in data if condition]

**List with transformation**

[x * 2 for x in data]

**Set**

{x for x in data}

**Set with condition**

{x for x in data if condition}

**Dictionary**

{x: x**2 for x in data}

**Dictionary with condition**

{x: x**2 for x in data if x > 5}

## *Key Takeaway*

Students should remember the three major patterns:

LIST

[expression for item in data]

↓

Creates a LIST

SET

{expression for item in data}

↓

Creates a SET

↓

Unique values

3DWEBSOFT FOUNDATION

DICTIONARY

{key: value for item in data}

↓

Creates a DICTIONARY

↓

Key → Value

## *Final Concept*

**Comprehension = Collect → Process → Filter → Create New Data**

This class is an important bridge from basic Python programming to **real data processing**, because students will later use the same ideas when working with **NumPy, Pandas, Machine Learning datasets, and AI applications**.

3DWEBSOFT FOUNDATION